

[TOC]

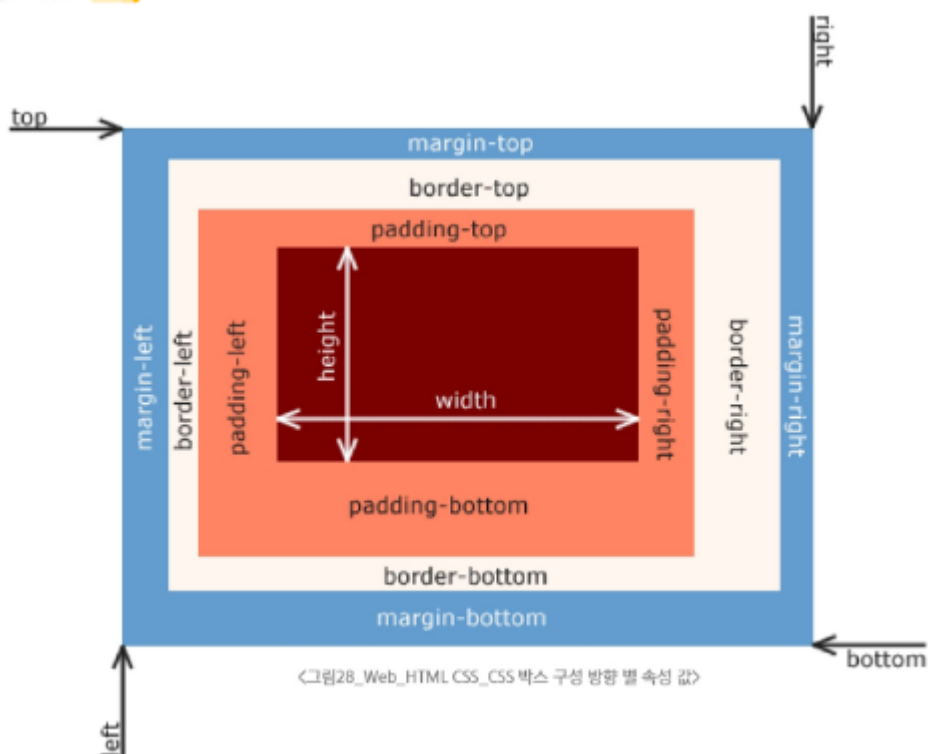
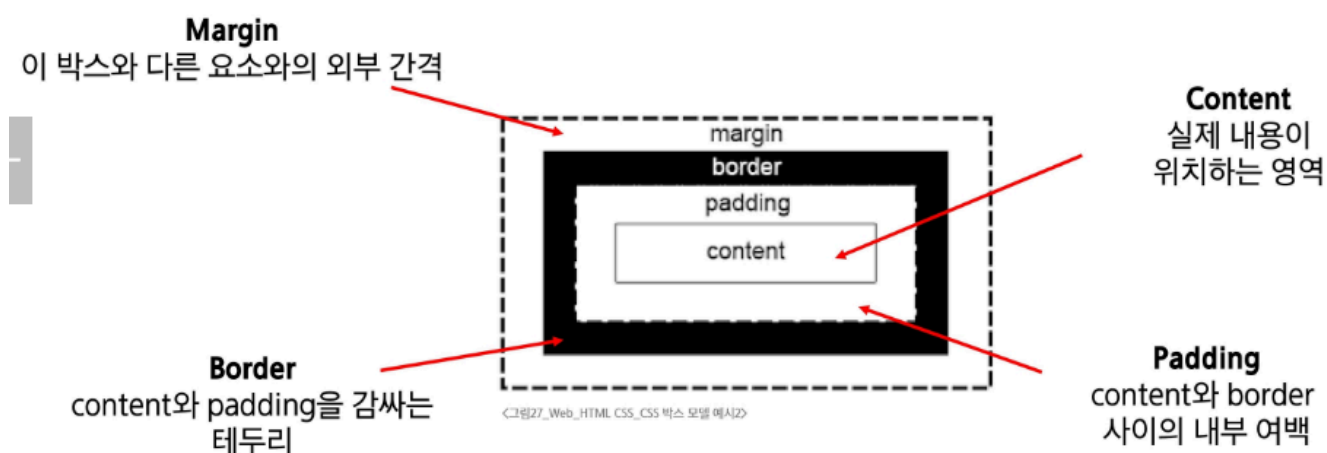
CSS layout

CSS Box Model

웹 페이지의 모든 HTML 요소를 감싸는 사각형 상자 모델

- 요소의 크기, 배치, 간격을 결정하는 규칙
- 원은 네모 박스를 깎은 것

박스 구성 요소



- width 조정은 박스의 전체 크기가 아니라 박스 안의 content 조정이다.

- 예시, padding-left: 25는 오른쪽으로 25 이동
- 박스 구성 요소는 F12 -> computed로 확인 가능

참고) web > 02_box_model
- 01_part_of_box.html

shorthand 속성 (단축 속성)

1. border

- border-width, border-style, border-color를 한번에 설정하기 위한 속성
- 작성 순서는 영향을 주지 않음
- border: 2px solid black;

2. 'margin' & 'padding'

```
/* 4개 - 상우하좌 */
margin: 10px 20px 30px 40px;
padding: 10px 20px 30px 40px;

/* 3개 - 상/좌우/하 */
margin: 10px 20px 30px;
padding: 10px 20px 30px;

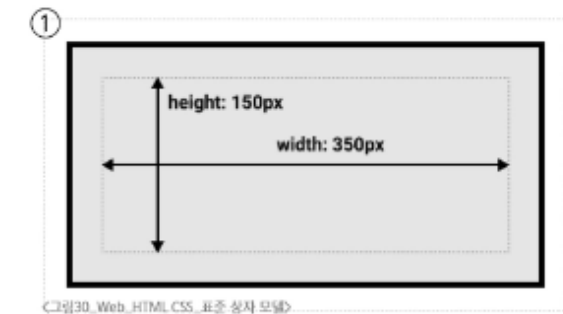
/* 2개 - 상하/좌우 */
margin: 10px 20px;
padding: 10px 20px;

/* 1개 - 공통 */
margin: 10px;
padding: 10px;
```

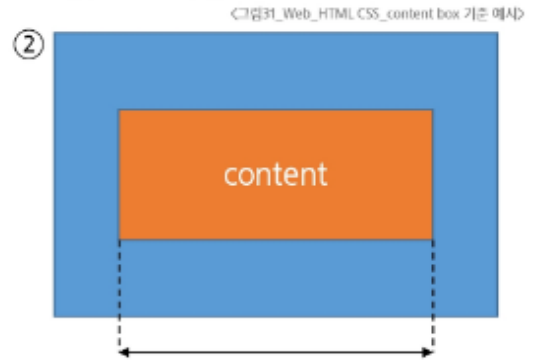
box-sizing 속성 (박스의 크기 계산법)

표준 상자 모델, The standard CSS box model (기본값: content-box)

- width와 height는 content 영역만의 크기
 - ① 표준 상자 모델에서 width와 height 속성 값을 설정하면, 이 값은 content box의 크기를 조정하게 됨
 - 요소가 실제로 차지하는 가로 크기 = width + padding(좌 + 우) + border(좌 + 우)
 - ② CSS는 border box가 아닌 content box의 크기를 width 값으로 지정
 - ** 콘텐츠 기준으로 설정된 값이라서 사람이 생각하는 크기가 아님

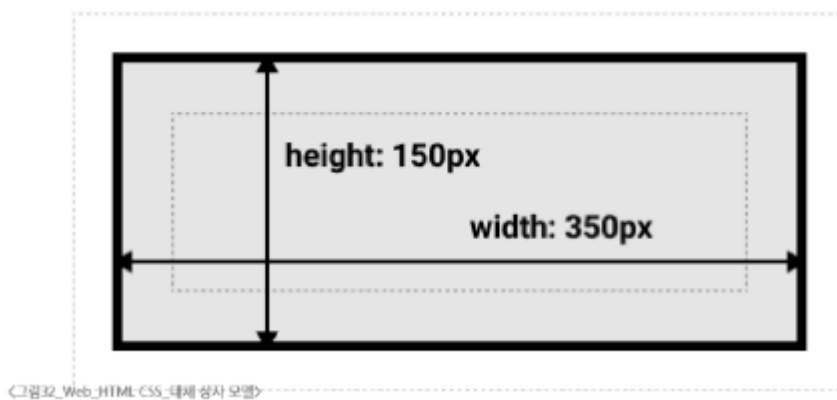


실제 박스 크기는 350px이 아닌 테두리, 패딩 값을 모두 더한 값을 가지게 됨

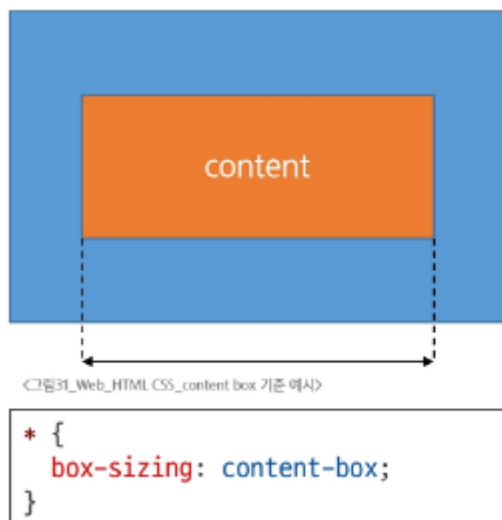


대체 상자 모델, The alternative CSS box model (**border-box**)

- **width**와 **height**가 **border**까지 포함한 크기, **padding**과 **border**는 그 안에서 자리를 차지
- 대체 상자 모델에서 모든 **width**와 **height**는 실제 상자의 너비
- 실제 박스 크기를 정하기 위해 테두리와 패딩을 조정할 필요 없음



대체 상자 모델 변경



- `<div class="box content-box">`

- class box로 규격 설정
- content-box로 모델 설정

표준 vs 대체 상자 모델

구분	표준 (content-box)	대체 (border-box)
width가 가리키는 것	content만	content + padding + border
width: 100px + padding 10px + border 2px	실제 124px	실제 100px
padding을 늘리면	상자가 바깥으로 커짐	안쪽 content가 줄어듦
기본 적용 여부	모든 요소의 기본값	직접 지정해야 함
사용하기 좋은 상황	내용 영역 크기를 정확히 맞춰야 할 때	레이아웃 칸 나누기, 반응형 배치

참고) web > 02_box_model
- 02_box_sizing.html

display 속성 (박스의 화면 배치 방식)

- 박스타입
 - 박스 타입에 따라 페이지에서의 배치 흐름 및 다른 박스와 관련하여 박스가 동작하는 방식이 달라짐
- 박스 타입 종류: Block 타입, Inline 타입

block 타입

하나의 독립된 덩어리처럼 동작하는 요소

- block 타입은 책의 각 문단과 같습니다.
- 모든 문단은 항상 새로운 줄에서 시작하며, 그 자체로 하나의 독립된 덩어리를 이룹니다. 다른 문단이 옆에 끼어들 수 없죠.
- 이처럼 block 타입은 웹 페이지의 큰 구조와 단락을 만듭니다.

```
.index {
  display: block;
}
```

block 특징

- 항상 새로운 행으로 나눔 (한 줄 전체를 차지, 너비 100%)
- `width`, `height`, `margin`, `padding` 속성을 모두 사용할 수 있음
- `padding`, `margin`, `border`로 인해 다른 요소를 상자로부터 밀어냄
- `width` 속성을 지정하지 않으면 박스는 inline 방향으로 사용 가능한 공간을 모두 차지함
 - 상위 컨테이너 너비 100%로 채우는 것
- 대표적인 block 타입 태그
 - `h1~h6`, `p`, `div`, `ul`, `li`

대표적인 태그: `<div>`

- 다른 HTML 요소들을 그룹화하여 레이아웃을 구성하거나 스타일링을 적용할 수 있음
- 헤더, 푸터, 사이드바 등 웹 페이지의 다양한 섹션을 구조화 하는 데 가장 많이 쓰이는 요소

```
<div class="container">
  <h1>제목</h1>
  <p>단락 내용입니다.</p>
</div>
<div>
  <p>콘텐츠</p>
</div>
```

제목

단락 내용입니다.

콘텐츠

Inline 타입

문장 안의 단어처럼 흐름에 따라 자연스럽게 배치되는 요소

- inline 타입은 문장 속 단어를 형광펜으로 칠하는 것과 같습니다.

- inline 타입은 줄을 바꾸지 않고, 텍스트의 **일부에만** 다른 스타일을 적용할 때 사용됩니다.

```
.index {
  display: inline;
}
```

inline 특징

- 줄 바꿈이 일어나지 **않음** (콘텐츠의 크기만큼만 영역을 차지)
- `width`와 `height` 속성을 사용할 수 없음
- 수직 방향 (상하)
 - `padding`, `margin`, `border`가 적용되지만, 다른 요소를 밀어낼 수는 없음
- 수평 방향 (좌우)
 - `padding`, `margin`, `border`가 적용되어 다른 요소를 밀어낼 수 있음
- 대표적인 inline 타입 태그
 - `a`, `img`, `span`, `strong`

2.3 대표적인 태그: `<span>`

- 자체적으로 시각적 변화 없음
 - 스타일을 적용하기 전까지는 특별한 변화 없음
- 텍스트 일부 조작
 - 문장 내 특정 단어나 구문에만 스타일을 적용할 때 유용
- 블록 요소처럼 줄 바꿈을 일으키지 않으므로, 문서의 구조에 큰 변화를 주지 않음

```
<p>이 문장에서 <span style="color: blue;">파란색</span> 단어만 색상이 다릅니다.</p>
<p>이 단어는 <span class="highlight-text">강조</span>되었습니다.</p>
<p>이것은 <span id="changeText">클릭</span>하면 변경됩니다.</p>
```

이 문장에서 파란색(파랑색임) 단어만 색상이 다릅니다.

이 단어는 강조되었습니다.

이것은 클릭하면 변경됩니다.

Normal flow

일반적인 흐름 또는 레이아웃을 변경하지 않은 경우 웹 페이지 요소가 배치되는 방식

위에서 아래로 왼쪽에서 오른쪽으로

- 워드의 경우 문단을 나누면 block 엔터 안누르고 타이핑하면 inline
 - block(구조조정) : 한 줄 전체
 - inline(세부조정) : 콘텐츠만큼의 공간만 차지 줄바꿈x

기타 display 속성

1. inline-block 타입

inline 과 block 특징을 모두 가진 특별한 display 속성 값

```
.index {  
  display: inline-block;  
}
```

특징

- block과 inline의 특징을 합친 것 (줄바꿈X, 크기 지정 가능)
- width 및 height 속성 사용 가능
- padding, margin 및 border로 인해 다른 요소가 상자에서 밀려남
 - 주로 가로로 정렬된 내비 메뉴나 여러 개의 버튼, 이미지 갤러리처럼 수평으로 나열하면서, 각 항목의 크기를 직접 제어하고 싶을 때 매우 용하게 사용합니다.

inline-block 타입 예시

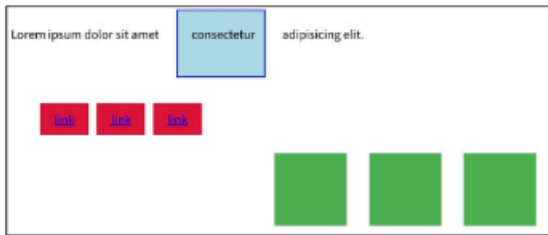


그림5_Web_CSS Layout_inlne-block 예시

```
<!-- 1. 이제 다른 요소를 밀어낼 수 있는 span -->
<p>Lorem ipsum dolor sit amet <span>consectetur</span> adipiscing elit. </p>
<!-- 2. 리스트 요소를 가로로 정렬 -->
<ul>
<li><a href="#">link</a></li>
<li><a href="#">link</a></li>
<li><a href="#">link</a></li>
</ul>
<!-- 3. div 요소를 가로로 정렬 -->
<div class="container">
  <div class="box"></div>
  <div class="box"></div>
  <div class="box"></div>
</div>
```

```
span {
  margin: 20px;
  padding: 20px;
  width: 80px;
  height: 50px;
  background-color: lightblue;
  border: 2px solid blue;
  display: inline-block;
}
ul>li {
  background-color: crimson;
  padding: 10px 20px;
  display: inline-block;
}
.container {
  text-align: center;
}
.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  background-color: #4CAF50;
  margin: 10px;
}
```

2. none 타입

요소를 화면에 표시하지 않고, 공간조차 부여되지 않음

```
.index {
  display: none;
}
```

```
<div class="box"></div>
<div class="box none"></div>
<div class="box"></div>
```

```
.box {
  width: 100px;
  height: 100px;
  background-color: red;
  border: 2px solid black;
}
.none {
  display: none;
}
```

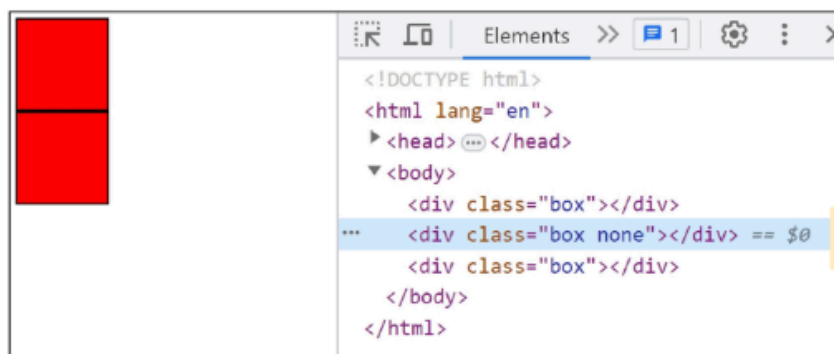


그림6_Web_CSS Layout_none 타입 예시

CSS Position

CSS Layout

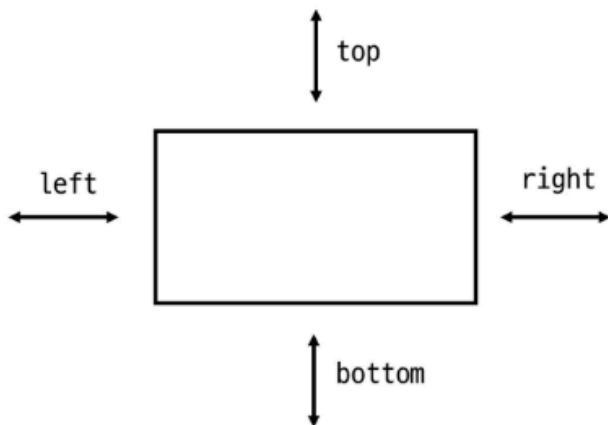
- 각 요소의 위치와 크기를 조정하여 웹 페이지의 디자인을 결정하는 것
- 요소들을 상하좌우로 정렬하고, 간격을 맞추고, 전체적인 페이지의 뼈대를 구성
- 핵심 속성: `display(block, inline, flex, grid, ...)`

CSS Position

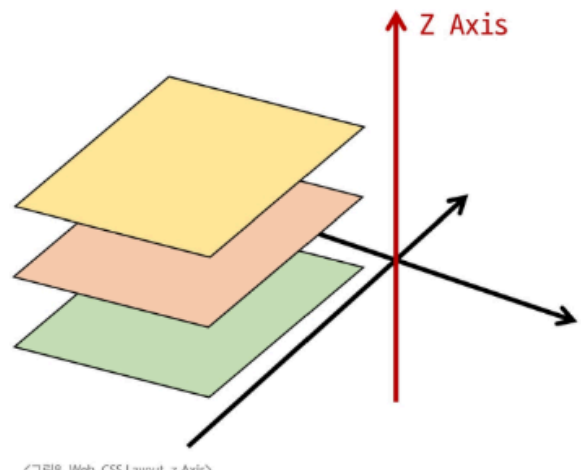
- 요소를 Normal Flow에서 제거하여 다른 위치로 배치하는 것
- 다른 요소 위에 올리기, 화면의 특정 위치에 고정시키기 등
- 핵심 속성: `position(static, relative, absolute, fixed, sticky, ...)`

position 이동 방향

- 네 가지 방향 속성(상, 하, 좌, 우)을 이용해 요소의 위치 조절 가능
- 겹치는 요소의 쌓이는 순서를 조절할 수 있음



<그림7_Web_CSS Layout_position 4가지 방향 속성>



<그림8_Web_CSS Layout_position 3D Axis>

포지션 유형

static, relative, absolute, fixed, sticky



<그림42_Web_CSS Layout_CSS Position 예시>

참고) web > 03_css_layout_position
- 01_posion.html

- 각 position 주석 풀면 박스 생김

1. static

- 요소를 normal flow에 따라 배치
- top, right, bottom, left 속성이 적용되지 않음
- 기본 값

```
.static {
  position: static;
  background-color: lightcoral;
}
```

2. relative

- 요소를 normal flow에 따라 배치
- 자신의 원래 위치(static)을 기준으로 이동
- top, right, bottom, left 속성으로 위치 조정
- 다른 요소의 레이아웃에 영향을 주지 않음
 - (요소가 차지하는 공간은 static일 때와 같음)

```
.relative {
  position: relative;
  background-color: lightblue;
  top: 100px;
  left: 100px;
  # 아래로 100, 오른쪽으로 100 이동
}
```



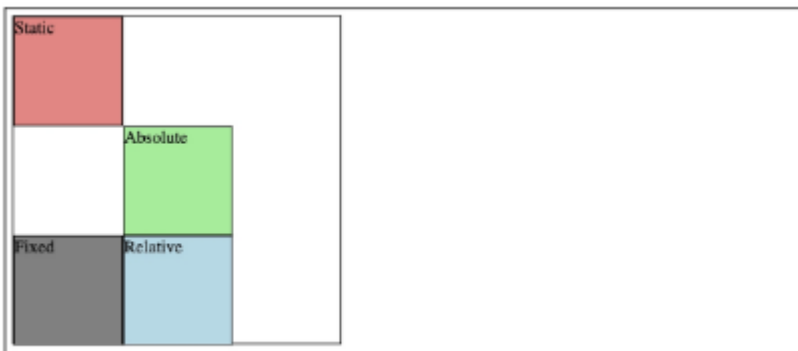
relative 박스가 이동해도 원래 본인의 과거 위치는 그대로 유지
= 빈 공간이 아니다.

= 이미지 메모 위치가 채워져 있음

3. absolute

- 요소를 normal flow에 따라 제거
- 가장 가까운 relative 부모 요소를 기준으로 이동
 - 만족하는 부모 요소가 없다면 body 태그를 기준으로 함
- top, right, bottom, left 속성으로 위치 조정
- 문서에서 요소가 차지하는 공간이 없어짐

```
.absolute {
  position: absolute;
  background-color: lightgreen;
  top: 100px;
  left: 100px;
}
```



<그림44_Web_CSS Layout_CSS Position 예시>

예: 네이버 홈에서 중간 추천 탭의 재생 버튼이나
기사 이미지 내 카테고리 문구

참고) web > 03_css_layout_position
 - 03.absolute.html
 - absolute 활용

- 실제 포지션 활용 예시

특정 요소 위에 다른 요소를 겹쳐서 배치할 때 유용하게 사용 됨

Normal flow 에서 벗어나
부모를 기준으로 좌측 상단 모서리로 이동



4.fixed

- 요소를 normal flow에 따라 제거
- 현재 화면영역(viewport)을 기준으로 함
- 스크롤해도 항상 같은 위치에 유지됨
- top, right, bottom, left 속성으로 위치 조정
- 문서에서 요소가 차지하는 공간이 없어짐

```
.fixed {
  position: fixed;
  background-color: gray;
  top: 0;
  left: 0;
}
```



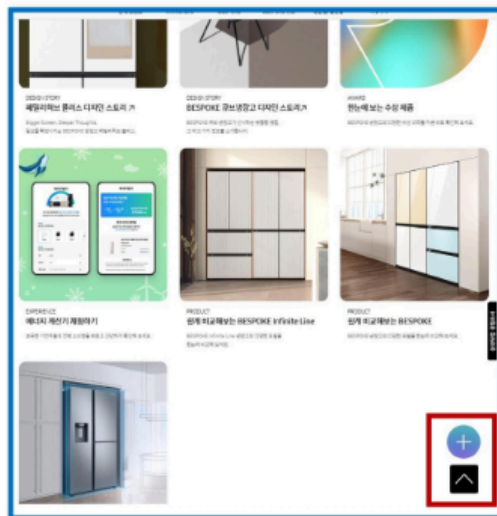
viewport 내가 보는 화면

→ 뷰포트 기준으로 고정을 시켜서 화면이 이동해도 고정된 위치에 계속 존재

예: 웹툰의 이동 화살표

- 실제 활용 예시

페이지를 스크롤해도 항상 같은 자리에 표시되는 요소를 만들 때 사용

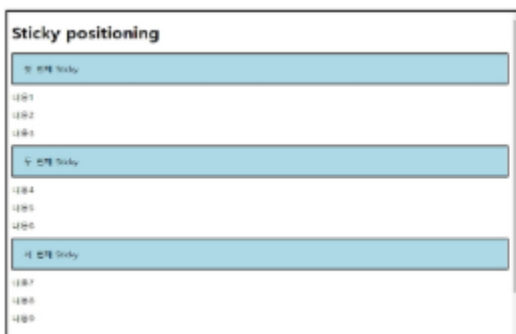


Normal flow에서 벗어나
화면을 기준으로 우측 하단에 위치

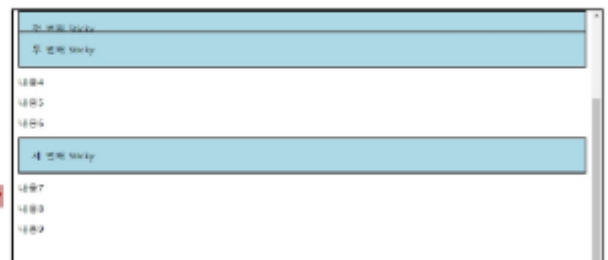
<그림34_Web_CSS Layout_fixed 활용>

5. sticky

- relative와 fixed의 특성을 결합한 속성
- 스크롤 위치가 임계점에 도달하기 전에는 relative처럼 동작
- 스크롤 위치가 임계점에 도달하면 fixed처럼 화면에 고정
- 다음 sticky 요소가 나오면 이전 sticky 요소의 자리를 대체
 - 이전 sticky 요소와 다음 sticky 요소의 위치가 겹치게 되기 때문



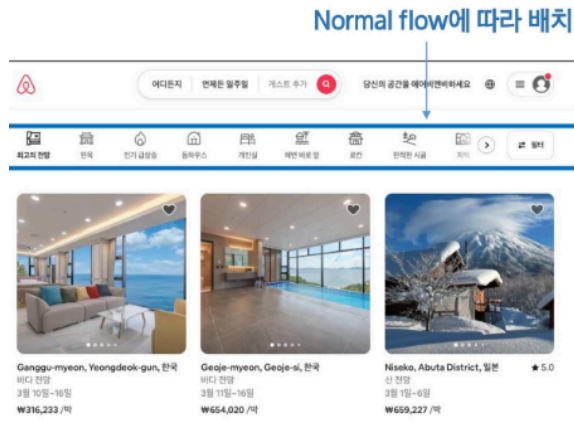
스크롤을 내렸을 때



<그림30_Web_CSS Layout_CSS Position sticky 예시>

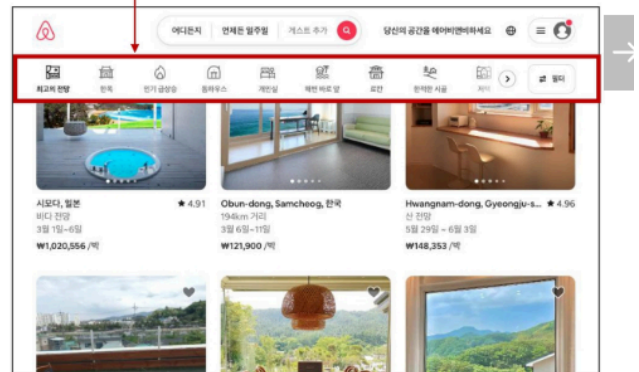
- 실제 활용 예시

- 일반적인 문서 흐름에 따라 배치되다가, 스크롤이 특정 위치에 도달하면 고정되는 속성



웹35_Web_CSS Layout_sticky 활용>

임계점에 도달한 이후 고정됨



참고) web > 03_css_layout_position
- 02.stiky.html

z-index

요소의 쌓임 순서를 정의하는 속성

- layer = z-index -> 시트가 많다

```
.index {
  z-index: 1;
}
```

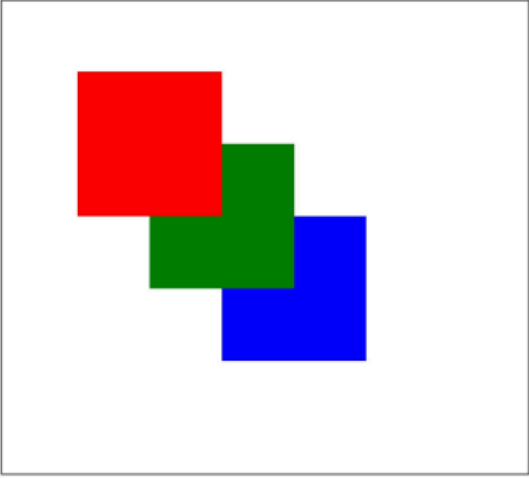
z-index 특징

- 정수 값을 사용해 Z축 순서를 지정
- 값이 클수록 요소가 위에 쌓이게 됨
- static이 아닌 요소에만 적용됨
- 기본값은 auto로 부모 요소의 z-index 값에 영향을 받음
- 같은 부모 내에서만 z-index 값을 비교하고, 값이 같으면 HTML 문서 순서대로 쌓임
- 부모의 z-index가 낮으면 자식의 z-index가 아무리 높아도 부모보다 위로 올라갈 수 없음

TIP

- position 속성이 static(기본값)이 아닌 요소에만 z-index가 적용됩니다.

- 음수 **z-index** 값은 요소를 부모 요소의 뒤(배경)로 보낼 때 사용할 수 있습니다.



```

<div class="container">
  <div class="box red"></div>
  <div class="box green"></div>
  <div class="box blue"></div>
</div>

.container {
  position: relative;
}
.box {
  position: absolute;
  width: 100px;
  height: 100px;
}

.red {
  background-color: red;
  top: 50px;
  left: 50px;
  z-index: 3;
}
.green {
  background-color: green;
  top: 100px;
  left: 100px;
  z-index: 2;
}
.blue {
  background-color: blue;
  top: 150px;
  left: 150px;
  z-index: 1;
}

```

- 원래는 파랑이초록위, 초록이 빨강 위에 있어야 함 이건 반대

참고) web > 03_css_layout_position
- 04.z_index.html

CSS Flexbox

제공해주신 이미지들의 내용을 정리한 마크다운 문서입니다.

박스 표시(Display) 타입

1. Outer display 타입

- block 타입
- inline 타입

2. Inner display 타입

- 박스 내부의 요소들이 어떻게 배치될지를 결정
- CSS Flexbox (속성: **flex**)

CSS Flexbox

요소를 행과 열 형태로 배치하는 1차원 레이아웃 방식

특징 및 개념

- 부모 컨테이너 필요, 그 안에 움직이는 아이탬 존재 (아이탬을 컨테이너가 조정)
- Flexbox(flex) 타입은 책장의 책들을 정리하는 것과 같습니다.
- 책장(Flex 컨테이너) 안에 책들(Flex 아이탬)을 넣고, "책들을 왼쪽에 붙여줘", "서로 같은 간격으로 띄워줘"와 같은 명령을 통해 손쉽게 배치할 수 있습니다.
- 큰 영역의 레이아웃과 배치 및 정렬만 가능 (세부 조정은 못 함)

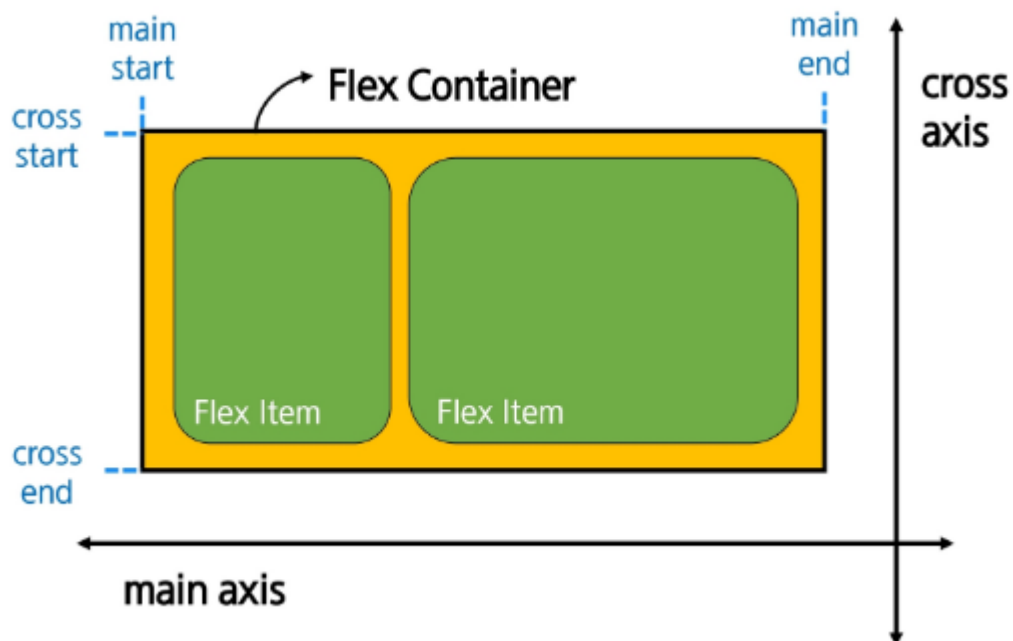
```
.container {
  display: flex;
}
```

'공간 배열' & '정렬'

- 요소를 행(Row) 또는 열(Column) 방향으로 화살표 축 기준에 맞춰 배치하는 구조를 가집니다.

Flexbox 구성요소

main axis, cross axis, flex container, flex item



1. main axis (주 축)

- flex item들이 배치되는 기본 축
- main start에서 시작하여 main end 방향으로 배치 (기본 값)
 - 메인 축을 세우면 교차축이 가로가 됨 = 변경 가능

2. cross axis (교차 축)

- main axis에 수직인 축
- cross start에서 시작하여 cross end 방향으로 배치 (기본 값)

3. Flex Container

- `display: flex;` 혹은 `display: inline-flex;` 가 설정된 부모 요소
- 이 컨테이너의 1차 자식 요소들이 Flex Item이 됨
- flexbox 속성 값들을 사용하여 자식 요소 Flex Item들을 배치하는 주체
- 기준 위치에 따라 내가 컨테이너 혹은 아이템이 될 수 있기 때문에 1차 자식 요소들이 아이템이 된다 그 1차 아이템 속에는 물론 또 다양한 아이템 존재 가능함

4. Flex Item

- Flex Container 내부에 레이아웃 되는 항목
- 이후 배우는 내용을 이용해 자유로운 순서 변경 및 정렬 가능

Flexbox 속성

Flex Container 관련 속성

- `display`
- `flex-direction`
- `flex-wrap`
- `justify-content`
- `align-items`
- `align-content`

Flex Item 관련 속성

- `align-self`
- `flex-grow`
- `flex-basis`
- `order`

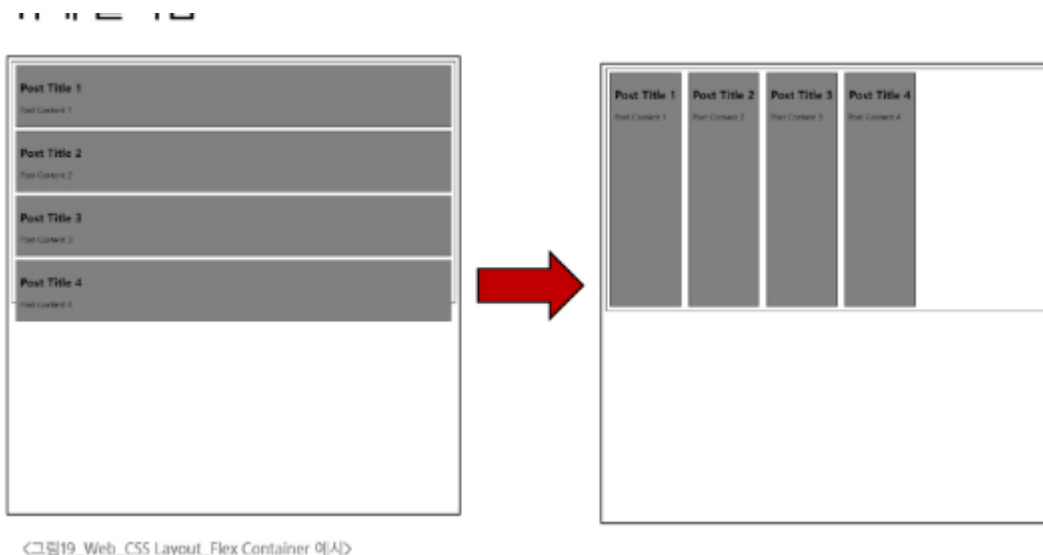
참고) web > 04_css_layout_flexible_box
- 01_flexbox.html

1. Flex Container 지정

메모 교차 축의 크기를 채우며, 행을 기준으로 하기 때문에 전에서 후로 변환

- `display` 속성을 `flex` 로 설정하면, Flex Container로 지정됨
- flex item은 기본적으로 행(주 축의 기본값인 가로 방향)으로 나열
- flex item은 주 축의 시작 선에서 시작
- flex item은 교차 축의 크기를 채우기 위해 늘어남

```
.container {
  height: 500px;
  border: 1px solid black;
  display: flex;
}
```



2. flex-direction

- flex item이 나열되는 방향을 지정

속성

- `row` (기본값): 아이템을 가로 방향으로, 왼쪽에서 오른쪽으로 배치
- `column`: 아이템을 세로 방향으로, 위에서 아래로 배치
- `"-reverse"`로 지정하면 flex item 배치의 시작 선과 끝 선이 서로 바뀜

▶ 상세 속성 이미지

```
.container {
  /* flex-direction: row; */
  flex-direction: column;
  /* flex-direction: row-reverse; */
  /* flex-direction: column-reverse; */
}
```



◀그림20_Web_CSS Layout_flex-direction 예시▶

3. flex-wrap

내용 Wrap의 경우 이미지와 같음

wrap-reverse인 경우 4가 위로 올라가고 123이 아래로 내려감

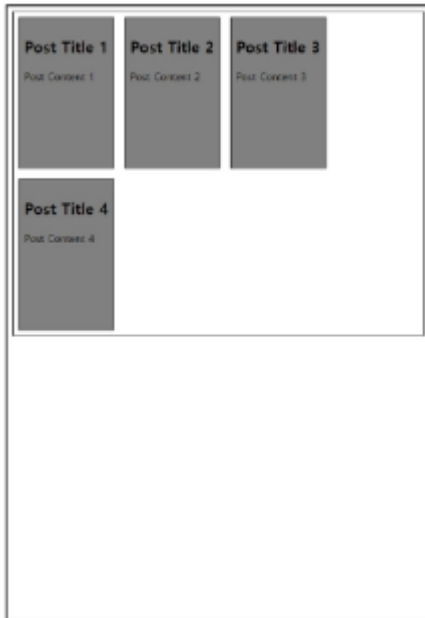
- flex item 목록이 flex container의 한 행에 들어가지 않을 경우, 다른 행에 배치할지 여부 설정

속성

- nowrap** (기본 값): 줄 바꿈을 하지 않음
- wrap**: 여러 줄에 걸쳐 배치될 수 있게 설정 (위에서 아래로 쌓임)
- wrap-reverse**: 여러 줄에 걸쳐 배치되나, 줄이 쌓이는 방향이 반대(역순)로 설정

▶ 상세 속성 이미지

```
.container {
  /* flex-wrap: nowrap; */
  flex-wrap: wrap;
  /* flex-wrap: wrap-reverse; */
} # 화면 너비를 줄여서 확인하기
```



<그림21_Web_CSS Layout_flex-wrap 예시>

4. justify-content

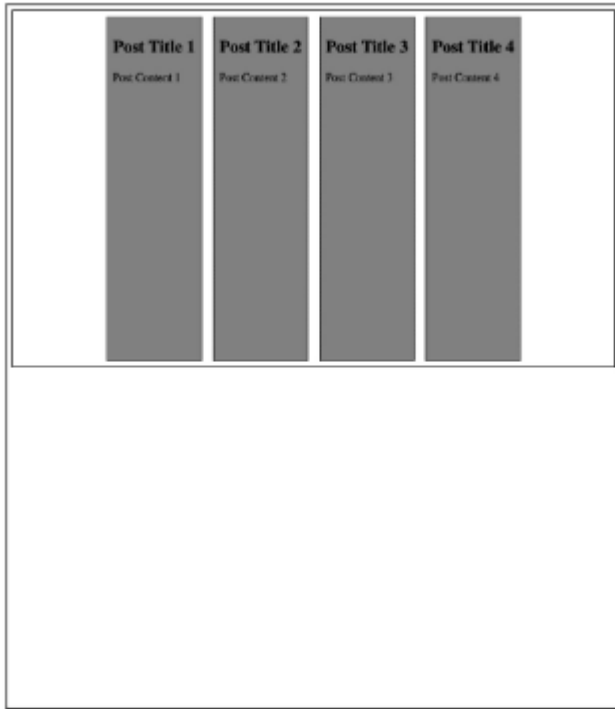
- 주 축을 따라 **flex item** 들을 정렬하고 간격을 조정

속성

- **flex-start** (기본값): 주 축의 시작점으로 정렬
- **center**: 주 축의 중앙으로 정렬
- **flex-end**: 주 축의 끝점으로 정렬

▶ 상세 속성 이미지

```
.container {
  /* justify-content: flex-start; */
  justify-content: center;
  /* justify-content: flex-end; */
}
```



〈그림22_Web_CSS Layout_justify-content 예시〉

5. align-content

메모

flex wrap이 있을 때 여러 줄을 기준으로 ! 교차 축 기준 한 줄일 때도 되긴 함

- 컨테이너에 여러 줄의 flex item이 있을 때, 그 줄들 사이의 공간을 어떻게 분배할지 지정
- **flex-wrap**이 **wrap** 또는 **wrap-reverse**로 설정된 여러 행에만 적용됨
 - **Flex** 아이템이 두 줄 이상일 때만 의미가 있음 (**flex-wrap**이 **nowrap**으로 설정된 경우 의미 없음)

속성

- **stretch** (기본값): 여러 줄을 교차 축에 맞게 늘려 빈 공간을 채움
- **center**: 여러 줄을 교차 축의 중앙에 맞춰 정렬
- **flex-start**: 여러 줄을 교차 축의 시작점(보통 위쪽)에 맞춰 정렬
- **flex-end**: 여러 줄을 교차 축의 끝점(보통 아래쪽)에 맞춰 정렬

▶ 상세 속성 이미지

```
.container {
  flex-wrap: wrap;

  /* align-content: flex-start; */
  align-content: center;
  /* align-content: flex-end; */
}
```



6. align-items

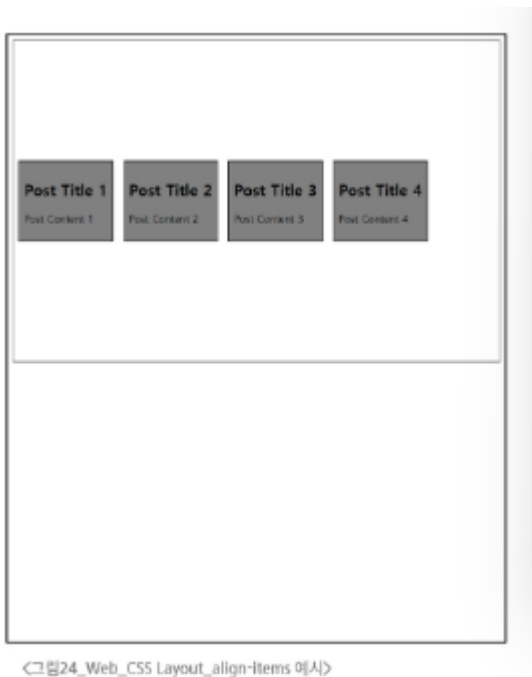
- 컨테이너 안에 있는 flex item 들의 **교차 축 정렬** 방법을 지정

속성

- stretch** (기본값): 아이템을 교차 축 높이를 꽉 채우도록 늘어남
- center**: 아이템을 교차 축의 중앙에 맞춰 정렬
- flex-start**: 아이템을 교차 축의 시작점(가로 방향일 경우 위쪽)에 맞춰 정렬
- flex-end**: 아이템을 교차 축의 끝점(가로 방향일 경우 아래쪽)에 맞춰 정렬

▶ 상세 속성 이미지

```
.container {
  align-items: center;
  /* align-items: stretch; */
  /* align-items: flex-start; */
  /* align-items: flex-end; */
}
```



7. align-self

메모

지금까지는 부모한테 속성을 줬는데 이것은 자식한테 줘야 함

- 컨테이너 안에 있는 **flex item** 들을 교차 축을 따라 개별적으로 정렬

속성

- auto** (기본값): 부모 컨테이너의 **align-items** 속성 값을 상속
- stretch**: 해당 아이템만 교차 축 방향으로 늘어나 컨테이너를 꽉 채우도록 정렬
- center**: 해당 아이템만 교차 축의 중앙에 정렬
- flex-start**: 해당 아이템만 교차 축의 시작점(가로 방향일 경우 위쪽)에 정렬
- flex-end**: 해당 아이템만 교차 축의 끝점(가로 방향일 경우 아래쪽)에 정렬

▶ 상세 속성 이미지

```
.item1 {
  align-self: center;
}

.item2 {
  align-self: flex-end;
}
```



<그림25_Web_CSS Layout_align-self 예시>

Flexbox 정리 & 요약

목적에 따른 속성 분류

- 배치: `flex-direction`, `flex-wrap`
- 공간 분배: `justify-content`, `align-content`
- 정렬: `align-items`, `align-self`

속성 쉽게 이해하는 방법

- `justify` - 주 축
- `align` - 교차 축

TIP

`justify-items` 및 `justify-self` 속성이 없는 이유는 뭘까요?

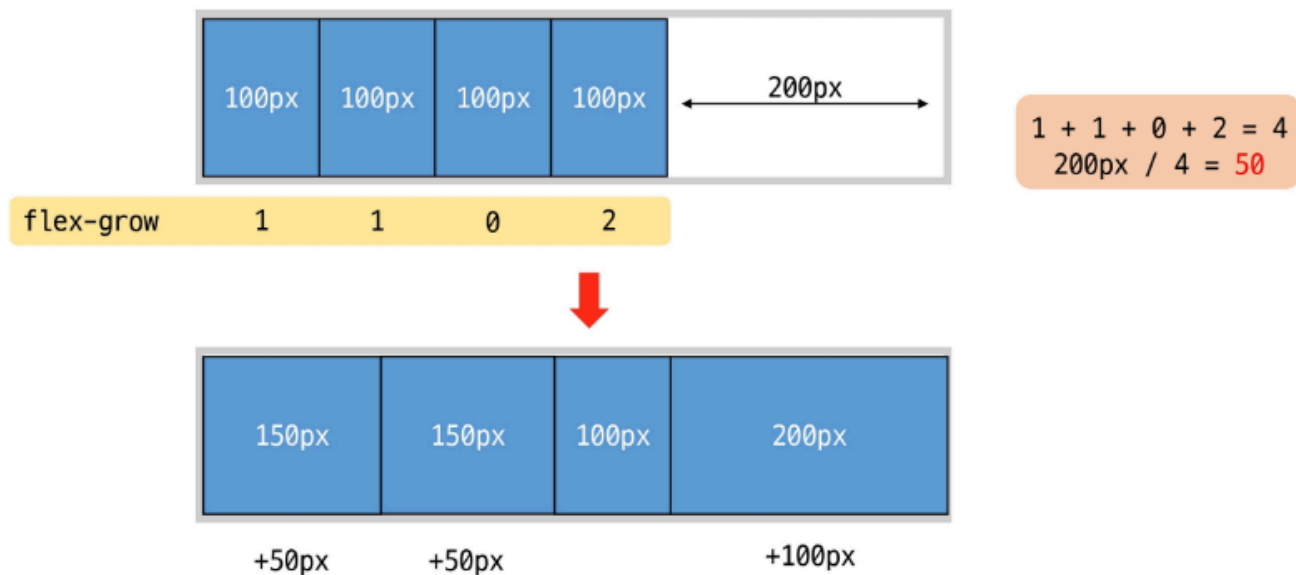
- 필요가 없기 때문입니다.
- 기존에 가지고 있는 기술인 `margin: auto`를 통해 정렬 및 배치가 가능합니다.

8. flex-grow

메모

언제나 꽉 찬 화면을 보여줄 수 있게 하는 것 비율 X -> grow가 몇개냐에 따라 등분이 몇개냐로 나뉜다

- 남는 행 여백을 비율에 따라 각 **flex item**에 분배
- flex item이 컨테이너 내에서 확장하는 비율을 지정



TIP: flex-shrink

- **flex-grow**의 반대되는 개념이에요.
- 컨테이너의 공간이 부족할 때, flex item이 줄어드는 비율을 지정하는 속성입니다.

참고) web > 04_css_layout_flexible_box
- 02_flexbox_grow.html

9. flex-basis

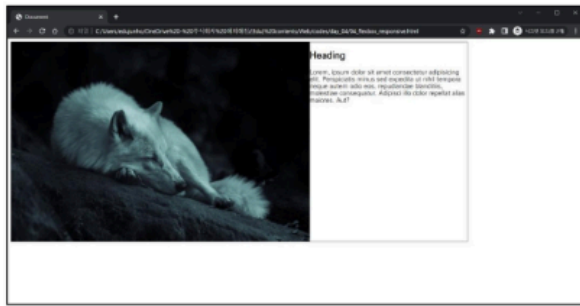
- flex item의 초기 크기 값을 지정
- flex-basis와 width 값을 동시에 적용한 경우 flex-basis가 우선

참고) web > 04_css_layout_flexible_box
- 02_flexbox_basis.html

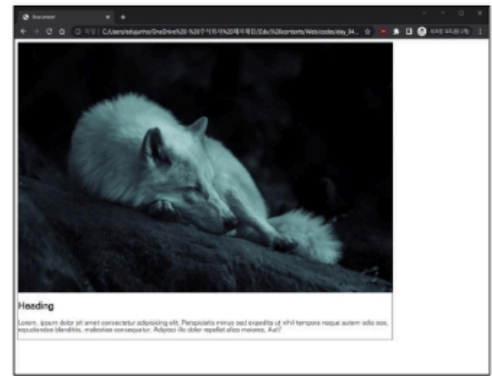
flex-wrap 응용

반응형 레이아웃 작성

- 다양한 디바이스와 화면 크기에 자동으로 적응하여 콘텐츠를 최적으로 표시하는 웹 레이아웃 방식
- **flex-wrap**을 사용해 반응형 레이아웃 작성 (**flex-grow** & **flex-basis** 활용)



창을 줄였을 때



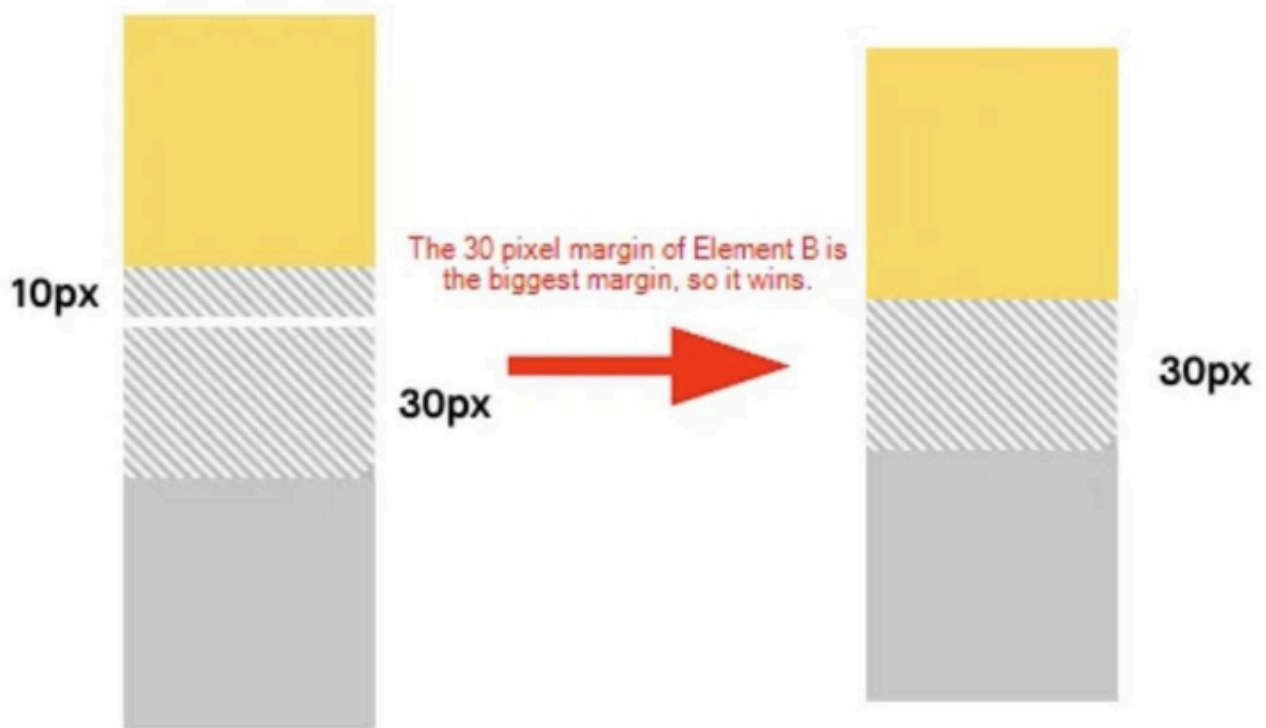
1. `.card` 요소를 flex 컨테이너로 설정
2. 컨테이너의 공간이 부족할 경우, 여러 줄로 나누어 배치되도록 허용
3. 각 flex item의 기본 너비를 설정
4. 컨테이너에 여유 공간이 있을 때 공간을 차지하며 늘어날 수 있도록 함 (두 flex item 모두 값이 1 이므로 절반씩 나누어 가짐)

참고) web > 04_css_layout_flexible_box
- 04_flexbox_responsive_layout.html

참고

마진 상쇄 (margin collapsing)

- 두 block 타입 요소의 margin top과 bottom이 만나 더 큰 margin으로 결합되는 현상



- 마진 상쇄 예시
 - 두 요소 모두 margin이 20px이지만 실제 두 요소의 상/하 공간은 40이 아닌 20으로 상쇄됨

TIP: 마진상쇄가 발생하는 이유

- 복잡한 레이아웃에서 요소 간 간격을 일관 되게 유지할 수 있습니다.(일관성)
- 요소 간의 간격을 더 예측 가능하고 관리하기 쉽게 만들 수 있습니다. (단순성)

이미지 속 내용을 마크다운 형식으로 정리해 드립니다.

박스 타입 별 수평 정렬

Block 요소의 수평 중앙 정렬

- `margin: auto` 사용
- 블록의 너비를 지정하고 좌우 마진을 `auto`로 설정

```
<div class="box margin-auto">
</div>
```

```
.box {
  width: 100px;
  height: 100px;
  background-color: crimson;
  border: 1px solid black;
```

```

}
.margin-auto {
  margin: 0 auto;
}

```

Inline 요소의 수평 중앙 정렬

- `text-align` 사용
- 부모 요소에 적용

HTML

```

<div class="text-center">
  <span>inline 요소</span>
</div>

```

CSS

```

.text-center {
  text-align: center;
}

```

Inline-block 요소의 수평 중앙 정렬

- `text-align` 사용
- 부모 요소에 적용

💡 **inline-block** inline과 block의 특징을 모두 가진 특별한 display 속성 값

HTML

```

<div class="text-center">
  <div class="box inline-block"></div>
</div>

```

CSS

```

.text-center {
  text-align: center;
}

```

```
.inline-block {
  display: inline-block;
}
```

Flexbox Shorthand 속성

이미지 속 내용을 깔끔하게 정리해 드립니다.

Shorthand: "flex-flow"

- `flex-direction`과 `flex-wrap` 속성을 한 번에 지정할 수 있는 단축 속성

CSS 예시

```
/* 기본 속성 사용 시 */
.container {
  flex-direction: row;
  flex-wrap: wrap;
}

/* 단축 속성 사용 시 */
.container {
  flex-flow: flex-direction flex-wrap;
}
```

Shorthand: "flex"

- `flex-grow`, `flex-shrink`, `flex-basis` 속성을 한 번에 설정할 수 있는 단축 속성 (기본값으로는 1, 1, 0%로 설정)

단축 속성 사용 vs 기본 속성 풀어서 표현

```
/* =====
1. One value, unitless number: flex-grow
===== */
/* 단축 속성 */
flex: 2;

/* 기본 속성 풀어서 표현 */
flex-grow: 2;
flex-shrink: 1;
flex-basis: 0%;
```

```
/* =====  
2. One value, length or percentage: flex-basis  
===== */  
/* 단축 속성 */  
flex: 10rem;  
flex: 30%;  
  
/* 기본 속성 풀어서 표현 */  
flex-grow: 1;  
flex-shrink: 1;  
flex-basis: 10rem; /* 또는 flex-basis: 30%; */  
  
/* =====  
3. Two values: flex-grow | flex-basis  
===== */  
/* 단축 속성 */  
flex: 1 30px;  
  
/* 기본 속성 풀어서 표현 */  
flex-grow: 1;  
flex-shrink: 1;  
flex-basis: 30px;  
  
/* =====  
4. Two values: flex-grow | flex-shrink  
===== */  
/* 단축 속성 */  
flex: 2 2;  
  
/* 기본 속성 풀어서 표현 */  
flex-grow: 2;  
flex-shrink: 2;  
flex-basis: 0%;  
  
/* =====  
5. Three values: flex-grow | flex-shrink | flex-basis  
===== */  
/* 단축 속성 */  
flex: 2 2 10%;  
  
/* 기본 속성 풀어서 표현 */  
flex-grow: 2;  
flex-shrink: 2;  
flex-basis: 10%;
```

마무리 정리

1. CSS Box Model - display 속성

웹 페이지에서 요소가 어떻게 보이고 다른 요소와 상호작용하는지를 결정합니다.

- **Block 타입**
 - 항상 새로운 줄에서 시작하며, 사용 가능한 전체 너비를 차지
 - 대표적인 태그: `<h1>`~`<h6>`, `<p>`, `<div>`, `<ul>`, `<li>` 등
 - **Inline 타입**
 - 새로운 줄에서 시작하지 않고, 콘텐츠의 너비만큼만 공간을 차지
 - 대표적인 태그: `<a>`, `<img>`, `<span>` 등
 - **inline-block 타입**
 - inline처럼 줄 바꿈 없이 다른 요소와 나란히 배치되지만, block처럼 `width`와 `height` 값을 지정할 수 있음
 - **none 타입**
 - 요소를 화면에 표시하지 않으며, 레이아웃에서 차지하는 공간도 없어짐
-

2. CSS Position

요소를 일반적인 흐름(Normal Flow)에서 벗어나 특정 위치에 배치하는 속성입니다.

- **static**
 - 모든 요소의 기본값으로, Normal Flow에 따라 배치
 - **relative**
 - 자신의 원래 위치(static 위치)를 기준으로 이동
 - 이동 후에도 원래 있던 공간은 그대로 차지
 - **absolute**
 - Normal Flow에서 완전히 벗어나, `position` 속성을 가진 가장 가까운 조상 요소를 기준으로 위치가 결정
 - **fixed**
 - absolute처럼 Normal Flow에서 벗어나지만, 뷰포트(브라우저 창)를 기준으로 위치가 고정
-

3. CSS Flexbox

기본 개념

- 요소를 행 또는 열의 1차원 형태로 배치하고 정렬하는 레이아웃 방식

핵심 구성 요소

- **Flex Container:** `display: flex;`가 적용된 부모 요소
- **Flex Item:** Flex Container의 직계 자식 요소들
- **main axis (주 축):** Flex Item들이 배치되는 기본 축
- **cross axis (교차 축):** 주 축에 수직인 축

Flex Container 관련 속성

- **flex-direction:** 아이템이 정렬될 주 축의 방향을 설정 (`row`, `column` 등)
- **flex-wrap:** 아이템이 한 줄에 들어가지 않을 때 줄 바꿈 여부를 결정 (`nowrap`, `wrap`)
- **justify-content:** 주 축 방향으로 아이템들의 정렬과 간격을 제어 (`flex-start`, `center`, `space-between` 등)
- **align-items:** 교차 축 방향으로 한 줄의 아이템들을 정렬 (`stretch`, `flex-start`, `center` 등)
- **align-content:** 여러 줄의 아이템들이 있을 때 교차 축 방향으로 줄들의 간격과 정렬을 제어

Flex Item 관련 속성

- **flex-grow:** 컨테이너에 여유 공간이 있을 때 아이템이 늘어나는 비율을 지정
- **flex-basis:** 아이템의 초기 크기를 설정
- **align-self:** 특정 아이템 하나만 개별적으로 교차 축 정렬을 변경할 수 있음

4. 핵심 키워드 정리

개념	설명	예시
display 속성	요소의 화면 배치 방식 정의	<code>.item { display: block; }</code>
position 속성	요소 위치를 특정 기준에 맞춰 배치	<code>.box { position: absolute; }</code>
z-index 속성	요소의 쌓이는 순서(Z축) 정의	<code>.box { z-index: 10; }</code>
CSS Flexbox	1차원 레이아웃 배치 및 정렬 방식	<code>.container { display: flex; }</code>
주 축 방향 설정	Flex 아이템이 나열될 방향 지정	<code>flex-direction: column;</code>
주 축 정렬	주 축의 아이템 정렬 및 간격 조정	<code>justify-content: center;</code>
align-items	교차 축의 아이템 한 줄 정렬	<code>align-items: center;</code>

? 확인 문제 및 정답/해설

1. 박스 모델에서 content 영역을 둘러싸고 있는 바깥 요소는?

- a) padding
- b) margin
- c) border
- d) display

2. 다음 중 블록 박스의 특징으로 올바른 것은?

- a) 같은 줄에 다른 요소를 배치할 수 있다
- b) 너비가 내용에 따라 결정된다
- c) 자동으로 줄 바꿈이 된다
- d) 텍스트만 포함할 수 있다

3. margin의 상하좌우 값을 한번에 설정 할 수 있는 shorthand 속성은?

- a) spacing
- b) area
- c) box-style
- d) margin

4. 다음 중 box-sizing: border-box 설정의 효과는?

- a) padding을 적용할 수 없다
- b) width에 padding과 border가 포함된다
- c) 요소가 자동으로 중앙 정렬된다
- d) margin이 포함된다

5. display 속성값 중 인라인과 블록 특성을 동시에 갖는 것은?

- a) inline
- b) block
- c) inline-block
- d) flex

6. 다음 중 position 속성이 아닌 것은?

- a) static
- b) fixed
- c) flexible
- d) absolute

7. position: relative를 사용하는 주된 이유는?

- a) 요소를 뷰포트에 고정한다
- b) 요소의 기준점을 설정한다
- c) 다른 요소를 덮어쓴다
- d) 중앙 정렬을 한다

8. z-index가 적용되지 않는 position 값은?

- a) absolute
- b) fixed
- c) relative
- d) static

9. Flexbox의 컨테이너 역할로 지정하는 속성은?

- a) inline

- b) float
- c) display: flex
- d) align-items

10. flexbox에서 메인 축(main axis)을 제어하는 속성은?

- a) justify-content
- b) align-items
- c) flex-direction
- d) flex-shrink

11. flex-wrap: wrap 설정 시의 효과는?

- a) 아이템들이 넘치더라도 한 줄에 유지된다
- b) 아이템들이 여러 줄로 자동으로 감싼다
- c) 아이템들이 사라진다
- d) flexbox가 적용되지 않는다

12. 마진 상쇄(Margin Collapse)가 발생하는 경우는?

- a) 일반적인 블록 형제 요소끼리
- b) position: absolute가 적용된 요소
- c) 인라인 요소 간
- d) float 요소 간

정답

1. a) padding
2. c) 자동으로 줄 바꿈이 된다
3. d) margin
4. b) width에 padding과 border가 포함된다
5. c) inline-block
6. c) flexible
7. b) 요소의 기준점을 설정한다
8. d) static
9. c) display: flex
10. a) justify-content
11. b) 아이템들이 여러 줄로 자동으로 감싼다
12. a) 일반적인 블록 형제 요소끼리

상세 해설

1. content 영역을 둘러싸고 있는 요소는 padding 입니다.
2. 블록 박스는 자동으로 줄 바꿈이 되며, 기본적으로 너비가 부모 요소를 채웁니다.
3. margin은 shorthand 속성으로 상하좌우 간격을 한 번에 설정할 수 있습니다.
4. border-box는 지정한 width 안에 padding과 border를 포함하여 계산합니다.
5. inline-block은 한 줄에 배치되면서도 width, height 설정이 가능합니다.
6. flexible은 유효한 position 속성이 아닙니다.

7. **relative**는 원래 위치 기준으로 이동하고, 자식 요소 **absolute**의 기준점을 제공합니다.
8. **static**인 요소에는 **z-index**가 적용되지 않습니다.
9. **display**를 **flex**로 설정해야 해당 요소는 **Flexbox** 컨테이너가 됩니다.
10. **justify-content**는 주 축 방향에서 항목을 정렬합니다.
11. **wrap**은 아이템들이 공간이 부족할 경우 자동으로 다음 줄로 넘어 가도록 합니다.
12. 마진 상쇄는 일반적인 문서 흐름에 있는 블록 레벨 요소의 상하 마진에서만 발생하며, **Float**, **Absolute**, **Flex**, **Grid**, 인라인 요소 등에서는 발생하지 않습니다.